

RCS Enterprise Modern - COGO Script Manual

This manual provides a comprehensive guide to formatting, writing, and executing **COGO Scripts** within the software. The COGO (Coordinate Geometry) Scripting Engine processes mathematical instructions to calculate points, lines, and figures precisely.

1. Engine Toggles

All scripts are assumed to start in COGO mode unless explicitly told otherwise. You can write mixed scripts (COGO + Piping). To ensure the compiler processes COGO sections correctly, use the engine toggles if returning from a piping block:

- **COGO-ENGINE-ON** : Ensures processing of geometry instructions.
 - **PIPE-ENGINE-OFF** : Pauses piping processing (useful for intermixing pure COGO code).
-

2. Direct Point Storage

The fastest way to build geometric nodes in a script is to store them explicitly using Northing, Easting, and Elevation values.

Store Point (2D)

Syntax: NE <NodeID> <Northing> <Easting> [Description] **Example:** NE 100 5000.0 5000.0 START_PT * Stores a 2D point at the specified Northing and Easting.

Store Point (3D)

Syntax: NEZ <NodeID> <Northing> <Easting> <Elevation> [Description] **Example:** NEZ 101 5000.0 5100.0 10.5 MH * Stores a 3D point including its elevation.

Standard Store

Syntax: ST <NodeID> <Northing> <Easting> <Elevation> [Description] **Example:** ST 102 5050.0 5100.0 12.0 Valve * An alias/alternative to NEZ for standard coordinate geometry workflows.

3. Survey Computations

You can calculate points sequentially relative to a "setup" coordinate (occupation) and an oriented "backsight" direction (azimuth/angle).

Occupy Point (OC)

Syntax: OC <NodeID> [InstrumentHeight] **Example:** OC 100 5.25 * Sets the current mathematical instrument location to the specified Node ID.

Backsight (BS)

Syntax: BS <NodeID> <Azimuth_DMS> **Example:** BS 101 45.3015 * Establishes the orientation of the instrument by defining the zero-angle direction towards a known point or a specified azimuth.

Foresight (FS)

Syntax: FS <NewNodeID> <Angle_DMS> <Distance> [Description] **Example:** FS 102 90.0000 150.5 CP * Calculates and stores a new point based on the angle turned from the backsight and the horizontal distance measured. *The instrument occupation stays at the current location.*

Traverse (TRAV)

Syntax: TRAV <NewNodeID> <Angle_DMS> <Distance> [Description] **Example:** TRAV 103 45.0000 200.0 TIE * Calculates and stores a new point exactly like FS, but **automatically moves the occupation (OC)** to the newly created point and sets the backsight to the previous point.

4. Analytical Tools

These commands evaluate relationships between existing points without creating new points. They output results to the master Command/Output Log.

Inverse (IN)

Syntax: IN <NodeID1> <NodeID2> **Example:** IN 100 101 * Returns the calculated bearing, horizontal distance, and vertical difference between two points.

Azimuth (AZ)

Syntax: AZ <NodeID1> <NodeID2> **Example:** AZ 100 101 * Returns the calculated absolute azimuth from node 1 to node 2.

5. Figures and Linework

To draw connecting lines between your points, use Figure direct commands. Figures interact seamlessly with the graphical canvas.

Begin Figure (B)

Syntax: B <NodeID> **Example:** B 100 * Starts a new linework figure at the given node.

Line-To (L)

Syntax: L <NodeID> **Example:** L 101 * Draws a segment from the active node to the next specified node.

Close Figure (C)

Syntax: C **Example:** C * Ends the active figure by drawing a line automatically back to the node specified in the B command.

6. Putting It All Together (Real-World Example)

Here is a snippet showing how to combine these rules into a working script.

```
``text // 1. Establish initial control NEZ 10 5000.0 5000.0 100.0 BASE NEZ 11 5100.0 5000.0 100.0 CTRL  
// 2. Setup your instrument OC 10 5.0 BS 11 0.0000 // Look toward pt 11  
// 3. Shoot some sideshots FS 12 45.3000 150.2 SW_CORNER FS 13 135.3000 200.5 SE_CORNER  
// 4. Draw a figure around them B 10 L 12 L 13 C  
// 5. Query Inverse IN 12 13 ``
```